Estratégias de Deploy e Limitação de Recursos

RollingUpdate, Blue/Green, Canary, Taints, Requests/Limits e HPA

Ada · DE-OP-011 Orquestração · Kubernetes | Aula 05 | Everton J Benedicto

Cronograma das Aulas

01
17/07 · Sexta
Conhecendo o Kubernetes

02
20/07 · Segunda
Mão na Massa: Minikube e kubectl

03
22/07 · Quarta
Pods, Namespaces e Controladores

04
24/07 · Sexta
Primeira Aplicação no K8S

05
27/07 · Segunda
Estratégias de Deploy e Limitação de Recursos

06
29/07 · Quarta
Services, Ingress e Volumes

07
31/07 · Sexta
Componentes e Troubleshooting

08
03/08 · Segunda
Cluster Multi-Master

09
05/08 · Quarta
Concluir atividades do módulo e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada = aula top!

INTERVALO • 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

TRABALHO FINAL

Entrega Final!

O desafio

- Suba no **minikube** um Pod com uma API e um Banco de Dados à sua escolha.
- A API deve fazer um **CRUD para produtos**.

TABELA produtos

Id	—	INT
Nome	—	VARCHAR(50)

O que significa CRUD

- **C** — **Create** — Criar
- **R** — **Read** — Ler
- **U** — **Update** — Atualizar
- **D** — **Delete** — Excluir

Atualize a versão do Pod de forma **declarativa**, demonstrando:

Rolling Update | **Recreate**

Objetivos de hoje

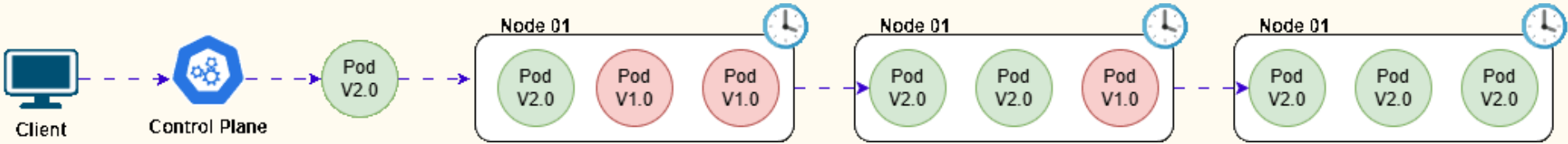
- Conhecer as estratégias de deploy do mercado
- Controlar a alocação de Pods com Taints e Tolerations
- Reservar e limitar recursos com requests e limits
- Escalar automaticamente com HPA
- Restringir recursos por namespace com LimitRange

1. Estratégias de Deploy

Reduzindo a indisponibilidade

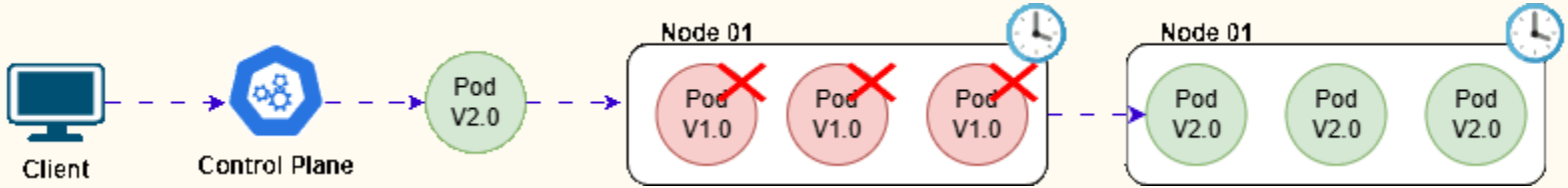
As quatro estratégias principais

- **RollingUpdate** — migração gradual e ordenada (PADRÃO do K8S)
  - ✓ **Como funciona**: O Kubernetes substitui os Pods da versão antiga pelos novos de forma progressiva (um por um ou em pequenos blocos)
  - ✓ **Vantagem**: Zero downtime - a aplicação não fica fora do ar em nenhum momento.
  - ✓ **Ponto de atenção**: Durante o processo, versões antigas e novas rodam ao mesmo tempo por alguns instantes.



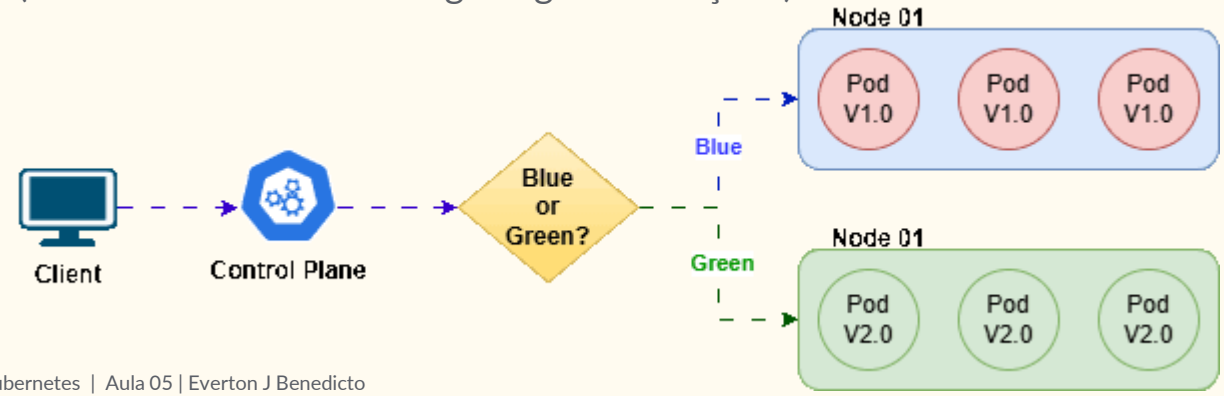
As quatro estratégias principais

- **Recreate** — derruba todos os Pods e recria com a nova versão
  - ✓ **Como funciona**: Todos os Pods da versão antiga são desligados de uma só vez antes que a nova versão seja iniciada.
  - ✓ **Vantagem**: Garante que duas versões diferentes nunca rodem juntas (necessário quando há mudanças incompatíveis no banco de dados, por exemplo).
  - ✓ **Ponto de atenção**: Causa downtime (o sistema fica fora do ar até a nova versão subir completamente).



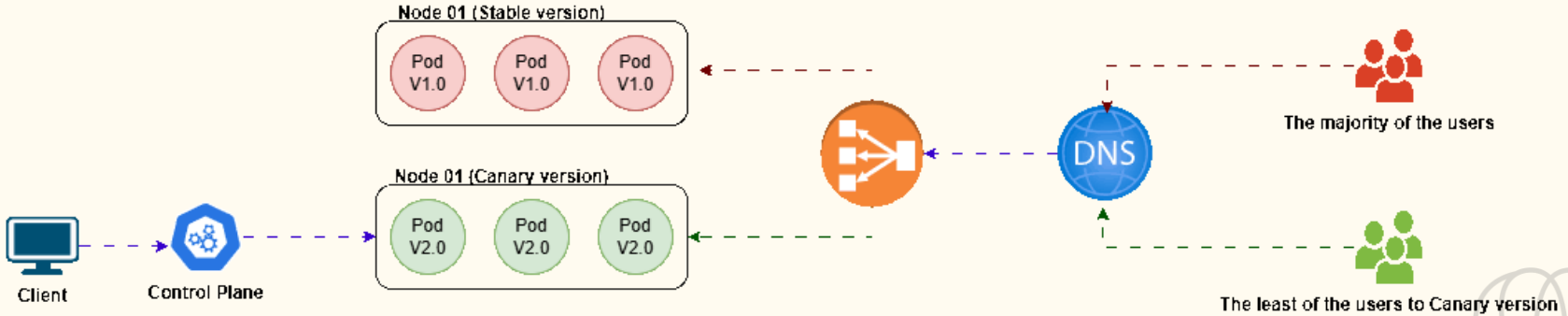
As quatro estratégias principais

- **Blue/Green** — dois ambientes idênticos; troca o tráfego após Validar
  - ✓ **Como funciona**: Mantém dois ambientes idênticos ativos: Blue (versão atual rodando) e Green (nova versão implantada). Após testar e validar o ambiente Green, o tráfego de todos os usuários é direcionado para ele de forma instantânea.
  - ✓ **Vantagem**: Rollback facílimo e imediato — se houver erro no Green, basta redirecionar o tráfego de volta para o Blue.
  - ✓ **Ponto de atenção**: Dobra o custo de infraestrutura temporariamente e exige ferramentas externas (como roteadores de tráfego/Ingress avançado).



As quatro estratégias principais

- **Canary** — libera a nova versão para uma fração dos usuários, gradualmente
  - ✓ **Como funciona**: A nova versão é liberada inicialmente apenas para uma pequena fatia de usuários (ex: 5% ou 10%). Conforme se valida a estabilidade, a porcentagem é aumentada gradualmente até cobrir 100%.
  - ✓ **Vantagem**: Minimiza riscos — se houver um bug, apenas um número reduzido de usuários será afetado.
  - ✓ **Ponto de atenção**: O Kubernetes padrão não faz o gerenciamento fino de tráfego do Canary por conta própria (exige ferramentas como Service Mesh/Istio).



As quatro estratégias principais

- **Recreate** — derruba todos os Pods e recria com a nova versão
  - ✓ **Como funciona**: Todos os Pods da versão antiga são desligados de uma só vez antes que a nova versão seja iniciada.
  - ✓ **Vantagem**: Garante que duas versões diferentes nunca rodem juntas (necessário quando há mudanças incompatíveis no banco de dados, por exemplo).
  - ✓ **Ponto de atenção**: Causa downtime (o sistema fica fora do ar até a nova versão subir completamente).
- Blue/Green — dois ambientes idênticos; troca o tráfego após validar
- Canary — libera a nova versão para uma fração dos usuários, gradualmente

Como escolher a estratégia

- Precisa de zero downtime e é stateless → RollingUpdate
- Pode ter downtime / mudança incompatível → Recreate
- Quer validar a nova versão em produção antes de trocar → Blue/Green
- Quer testar com poucos usuários e crescer aos poucos → Canary

2. Taints e Tolerations

Controlando onde os Pods rodam

Taints: repelindo Pods de um node

- Taint = marca no NODE que repele Pods sem a tolerância correspondente
- Evita conflitos e uso indevido de nodes específicos
- Efeitos: NoSchedule, PreferNoSchedule, NoExecute
  - NoSchedule: não agenda novos Pods sem toleration
  - NoExecute: remove Pods existentes sem toleration
  - PreferNoSchedule: evita agendar, mas não é regra rígida

Tolerations: a permissão de exceção

- Toleration = configuração no POD que permite rodar em node com taint
- Operador Equal: valor deve ser exatamente igual ao do taint
- Operador Exists: basta existir o taint com aquele efeito
- Taint + Toleration compatíveis = Pod pode rodar no node

Testando Taints e Tolerations

MÃO NA MASSA

1. Crie o namespace caso já não tenha criado

```
$ kubectl create namespace aplicação
```

2. Crie o Pod da aplicação feita em Ruby (primeira-aplicacao.yaml)

```
$ kubectl apply -f primeira-aplicacao.yaml
```

3. Aplique um taint que expulsa pods com label app=ruby:

```
$ kubectl taint nodes minikube app=ruby:NoExecute
```

4. Veja os pods entrarem em Terminating/Pending:

```
$ kubectl get pods -n aplicacao
```

5. Adicione o toleration ao Deployment e reaplique (primeira-aplicacao.yaml)

```
$ kubectl apply -f primeira-aplicacao.yaml
```

6. Para remover o taint, repita o comando com hífen ao final:

```
$ kubectl taint nodes minikube app=ruby:NoExecute-
```

Testando Taints e Tolerations

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl create namespace aplicacao
namespace/aplicacao created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl apply -f primeira-aplicacao.yaml
deployment.apps/primeira-aplicacao created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl pods get
error: unknown command "pods" for "kubectl"

Did you mean this?
        logs
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods
NAME                                READY   STATUS    RESTARTS        AGE
nginx                               1/1     Running   1 (8m16s ago)   22h
nginx-deployment-77bc6bd484-bqlhw   1/1     Running   1 (8m16s ago)   20h
nginx-deployment-77bc6bd484-dm7sl   1/1     Running   1 (8m16s ago)   20h
nginx-deployment-77bc6bd484-gjzs7   1/1     Running   1 (8m16s ago)   20h
nginx2                              1/1     Running   1 (8m16s ago)   22h
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods -n aplicacao
NAME                                  READY   STATUS    RESTARTS   AGE
primeira-aplicacao-564b64f5b4-5mg8c   1/1     Running   0          45s
primeira-aplicacao-564b64f5b4-7fftd   1/1     Running   0          45s
primeira-aplicacao-564b64f5b4-ms9pm   1/1     Running   0          45s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl taint nodes minikube app=ruby:NoExecute
node/minikube tainted
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods -n aplicacao
NAME                                  READY   STATUS    RESTARTS   AGE
primeira-aplicacao-564b64f5b4-5mg8c   0/1     Error     0          95s
primeira-aplicacao-564b64f5b4-6bspg   0/1     Pending   0          3s
primeira-aplicacao-564b64f5b4-7fftd   0/1     Error     0          95s
primeira-aplicacao-564b64f5b4-8kvq9   0/1     Pending   0          3s
primeira-aplicacao-564b64f5b4-xc6rg   0/1     Pending   0          3s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods -n aplicacao
NAME                                  READY   STATUS    RESTARTS   AGE
primeira-aplicacao-564b64f5b4-6bspg   0/1     Pending   0          15s
primeira-aplicacao-564b64f5b4-8kvq9   0/1     Pending   0          15s
primeira-aplicacao-564b64f5b4-xc6rg   0/1     Pending   0          15s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods -n aplicacao
NAME                                  READY   STATUS    RESTARTS   AGE
primeira-aplicacao-564b64f5b4-6bspg   0/1     Pending   0          26s
primeira-aplicacao-564b64f5b4-8kvq9   0/1     Pending   0          26s
primeira-aplicacao-564b64f5b4-xc6rg   0/1     Pending   0          26s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl apply -f primeira-aplicacao.yaml
deployment.apps/primeira-aplicacao unchanged
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl taint nodes minikube app=ruby:NoExecute-
node/minikube untainted
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods -n aplicacao
NAME                                  READY   STATUS    RESTARTS   AGE
primeira-aplicacao-564b64f5b4-6bspg   1/1     Running   0          2m5s
primeira-aplicacao-564b64f5b4-8kvq9   1/1     Running   0          2m5s
primeira-aplicacao-564b64f5b4-xc6rg   1/1     Running   0          2m5s
```

Fonte da imagem: Ada Tech

Toleration no manifesto

trecho adicionado ao spec.template.spec do Deployment

```
tolerations:
- key: "app"
  operator: "Equal"
  value: "ruby"
  effect: "NoExecute"
```

Testando Taints e Tolerations

MÃO NA MASSA

1. Aplique um taint que expulsa pods com label app=ruby :

```
$ kubectl taint nodes minikube app=ruby:NoExecute
```

2. Crie o Pod da aplicação feita em Ruby (primeira-aplicacao.yaml)

```
$ kubectl apply -f primeira-aplicacao.yaml
```

3. Crie o Pod da aplicação feita em Ruby (primeira-aplicação-toleration.yaml)

```
$ kubectl apply -f primeira-aplicacao-toleration.yaml
```

4. Veja os pods criados:

```
$ kubectl get pods -n aplicacao
```

Toleration no manifesto

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl taint nodes minikube app=ruby:NoExecute
node/minikube tainted
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl apply -f primeira-aplicacao.yaml
deployment.apps/primeira-aplicacao unchanged
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl apply -f primeira-aplicacao-toleration.yaml
deployment.apps/primeira-aplicacao configured
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get pods -n aplicacao
NAME                                 READY   STATUS    RESTARTS   AGE
primeira-aplicacao-6c5c445f44-8bcfn   1/1     Running   0          6s
primeira-aplicacao-6c5c445f44-s4sfr   1/1     Running   0          5s
primeira-aplicacao-6c5c445f44-zmdkp   1/1     Running   0          8s
```

Fonte da imagem: Ada Tech

3. Recursos: Requests e Limits

Reservar e limitar CPU/memória

Requests × Limits

Sem limites, o padrão é perigoso

Por padrão o container consome o quanto quiser. Um vazamento ou pico de memória pode esgotar o nó inteiro e derrubar as outras aplicações que rodam nele.

REQUESTS *garantia mínima*

- O mínimo que o container precisa para funcionar.
- O Scheduler olha só para os requests e escolhe um nó com pelo menos esse valor livre.

LIMITS *teto máximo*

- O máximo permitido, mesmo com sobra no nó.
- Memória acima do teto → **OOMKilled** (reiniciado).
- CPU acima do teto → **throttling** (fica lento).

Unidades **CPU:** 1 CPU = 1000m · 250m = 0,25 núcleo (25%). **Memória:** Mi (Mebibytes) / Gi (Gibibytes).

Na prática

Request → "o que preciso garantido para iniciar."

Limit → "o teto que nunca posso ultrapassar."

Mi e Gi: memória em base binária

Mi e Gi são unidades do **sistema binário** — Mebibytes e Gibibytes — que o Kubernetes usa para medir memória RAM.

Binário (Kubernetes)	Em bytes	Decimal (tradicional)	Em bytes
1 Ki Kibibyte	$2^{10} \rightarrow 1.024$	**1 KB** Kilobyte	$10^{3} \rightarrow 1.000$
1 Mi Mebibyte	$2^{20} \rightarrow 1.048.576$	**1 MB** Megabyte	$10^{6} \rightarrow 1.000.000$
1 Gi Gibibyte	$2^{30} \rightarrow 1.073.741.824$	**1 GB** Gigabyte	$10^{9} \rightarrow 1.000.000.000$

Por que o "i"?

O sistema operacional conta a RAM em potências de 2 (1024). O padrão IEC usa o "i" para marcar essa base binária e evitar ambiguidade com o decimal.

Na prática

- 1 Mi é ~ **4,8% maior** que 1 MB.
- 1 Gi é ~ **7,3% maior** que 1 GB.

Dica: nos manifests use sempre **`Mi`** ou **`Gi`** (ex.: **`512Mi`**, **`2Gi`**) — é a precisão exata que o SO usa para gerenciar a RAM.

requests e limits em um Pod

reserva 64Mi/250m, teto de 128Mi/500m

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: frontend
spec:
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

HPA — Horizontal Pod Autoscaler

- Escala o número de Pods automaticamente conforme a carga
- Métricas comuns: CPU e memória (ou métricas customizadas)
- Baseia-se no percentual sobre o limite definido
- Define mínimo e máximo de réplicas
- kubectl autoscale deployment ... --cpu-percent=50 --min=1 --max=10

Criando um HPA

MÃO NA MASSA

Crie o autoscaler para o Deployment:

```
$ kubectl autoscale deployment primeira-aplicacao -n aplicacao --cpu-percent=50 --min=1 --max=10
```

Verifique o HPA criado:

```
$ kubectl get hpa -n aplicacao
```

--cpu-percent=50: escala ao atingir 50% do limite de CPU

--min=1 / --max=10: limites inferior e superior de réplicas

Criando um HPA

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl autoscale deployment primeira-aplicacao -n aplicacao --cpu-percent=50 --min=1 --max=10
horizontalpodautoscaler.autoscaling/primeira-aplicacao autoscaled
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get hpa -n aplicacao
NAME                REFERENCE                      TARGETS             MINPODS   MAXPODS   REPLICAS   AGE
primeira-aplicacao   Deployment/primeira-aplicacao   cpu: <unknown>/50%   1         10        3          15s
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05>
```

Fonte da imagem: Ada Tech

LimitRange — limites por namespace

- Aplica mínimos e máximos de recursos por container/Pod no namespace
- Pode recusar Pods criados fora dos limites
- Pode definir requests/limits padrão para Pods que não os declararam
- Governança: impede abuso de recursos por qualquer aplicação

LimitRange — limites por namespace

Crie o arquivo limitrange.yaml

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-aplicacao
spec:
  limits:
  - max:
      cpu: "800m"
    min:
      cpu: "200m"
    type: Container
```

LimitRange — limites por namespace

Crie o arquivo pod-limit.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: limit-cpu
spec:
  containers:
  - name: limit-cpu
    image: nginx:1.14.2
    resources:
      limits:
        cpu: "1500m"
      requests:
        cpu: "100m"
```

LimitRange na prática

MÃO NA MASSA

Aplique um LimitRange (min 200m, max 800m de CPU) no namespace:

```
$ kubectl apply -f limitrange.yaml -n aplicacao
```

Inspecione o LimitRange:

```
$ kubectl get limitrange cpu-aplicacao -o yaml -n aplicacao
```

Tente criar um Pod fora dos limites (será recusado):

```
$ kubectl create -f pod-limit.yaml -n aplicacao
```

Erro esperado: request 100m < mínimo 200m; limit 1500m > máximo 800m

LimitRange na prática

```
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl apply -f limitrange.yaml -n aplicacao
limitrange/cpu-aplicacao created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl get limitrange cpu-aplicacao -o yaml -n aplicacao
apiVersion: v1
kind: LimitRange
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"LimitRange","metadata":{"annotations":{},"name":"cpu-aplicacao","namespace":"aplicacao"},"spec":{"limits":[{"max":{"cpu":"800m"},"min":{"cpu":"200m"},"type":"Container"}]
}}
  creationTimestamp: "2026-07-23T21:06:50Z"
  name: cpu-aplicacao
  namespace: aplicacao
  resourceVersion: "16868"
  uid: a9c3fd40-92e1-4a2d-8d87-77864df25bbd
spec:
  limits:
  - default:
      cpu: 800m
    defaultRequest:
      cpu: 800m
    max:
      cpu: 800m
    min:
      cpu: 200m
    type: Container
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05> kubectl create -f pod-limit.yaml -n aplicacao
Error from server (Forbidden): error when creating "pod-limit.yaml": pods "limit-cpu" is forbidden: [minimum cpu usage per Container is 200m, but request is 100m, maximum cpu usage per Container is 800m,
 but limit is 1500m]
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s\aula-05>
```

Fonte da imagem: Ada Tech

Recapitulando a Aula 05

- ✓ Estratégias: RollingUpdate, Recreate, Blue/Green, Canary
- ✓ Taints (repelem) + Tolerations (autorizam) controlam a alocação
- ✓ requests reservam, limits limitam — sempre defina em produção
- ✓ HPA escala Pods automaticamente; LimitRange governa o namespace
- ✓ Próxima aula: Services, Ingress e Volumes

